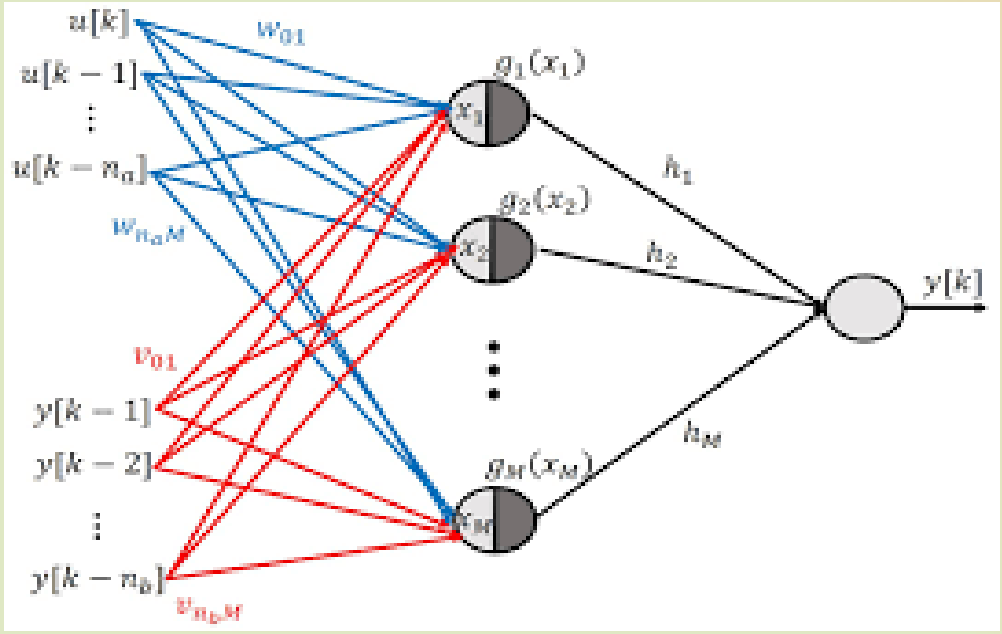
ARX Neliniar
Gr 11/1



Proiect – partea 2 : Almasan Antonio

Culda Alisa

Motoc Amelie

Descrierea problemei

~ dezvoltarea unui model folosind structura ARX neliniara

~ sistem cu o intrare si iesire

~ regresie pentru identificarea paramterilor

~ utilizarea modelului cu intrari noi

~ identificarea si valiadrea problemei pe date experimentale

Cand sa alegi ARX si cand Narx?

ARX LINIAR:

~ sisteme cu dinamica simpla

~ date limitate

~ neliniaritati semnificative

ARX NELINIAR:

~sisteme cu neliniaritati pronuntate

~ date suficiente

ARX

Caracteristici:

- formă liniară în variabile
- parametri: ai, bj
- număr parametri: na+nb
- dependenta liniara de iesirile si intrarile precedenre: y(k-1), … , y(k-na), u(k-1), …, u(k – nb);

$$y(k) = -a_1 y(k-1) - a_2 y(k-2) - \ldots - a_{na} y(k-na) + b_1 u(k-1) + b_2 u(k-2) + \ldots + b_{nb} u(k-nb) + e(k)$$

NARX

Caracteristici:

- formă polinomială de grad m
- include termeni neliniari
- număr mult mai mare de parametri

De exemplu pt na = nb = 1 si gradul m = 2:

y(k) =ay(k −1)+bu(k −1)+cy(k −1)^2 +du(k −1)^2 +wu(k −1)y(k −1)+z +e(k)

Descrierea structurii aproximatorului

$$y(k) = g(y(k-1), y(k-2), \ldots, y(k-na), u(k-1), u(k-2), \ldots, u(k-nb); \theta) + e(k)$$

Functia g este parametrizata de $\theta \in R^n$, iar acesti parametri pot fi alesi pentru a recupera datele de identificare, obtinand astfel un sistem dinamic neliniar.

~ g este un polinom de gradul m:

$y(k) = p(y(k-1), \ldots, y(k-na), u(k-1), \ldots, u(k-nb)) + e(k)$, unde $d(k) = [y(k-1), \ldots, y(k-na), u(k-1), \ldots, u(k-nb)]$ transpus este vectorul de semnale precedente.

Exemplu: na = nb = 1

m = 1 => $y(k) = c + ay(k-1) + bu(k-1) + e(k)$

m = 2 => $y(k) = ay(k-1) + bu(k-1) + cy(k-1)^2 + du(k-1)^2 + wu(k-1)y(k-1) + z + e(k)$

m = 3 => $y(k) = ay(k-1) + bu(k-1) + cy(k-1)^2 + du(k-1)^2 + wu(k-1)y(k-1) + dy(k-1)^3 + fu(k-1)^3 + g(u(k-1)^2 * y(k-1)) + h(u(k-1) * y(k-1)^2) + e(k)$

=> Pe masura ce creste m rezulta mai multi termeni

Metoda regresiei liniare pentru NARX

(determinarea parametrilor)

regresia liniara functioneaza ca de obicei;
gaseste parametrii care minimizeaza valoarea MSE-ului

$$y(k) = \varphi^{\top}(k)\theta + \varepsilon(k)$$

$\phi(k)$ – iesirile si intrarile din y
θ – coeficientii polinoamelor

Pentru a crea θ, se foloseste regresia liniara.
Cu ajutorul regresiei liniare care minimizeaza eroarea medie patratica $V(\theta)$ se obtine:

$$\widehat{\theta} = (\Phi^{\top}\Phi)^{-1}\Phi^{\top}Y \qquad V(\theta) = \frac{1}{N}\sum_{k=1}^{N}\varepsilon(k)^2$$

Caracteristici generale

~ separația clară: am impartit in functii pentru ca codul sa fie mai usor de inteles
~ consistenta: acelasi algoritm, atat in predictie cat si in simulare
~ gestionam marginea de la simulare, evitand eroarea de la inceput (k-1)>0
~ grafice bine explicate: fiecare figura ne ajuta sa intelegem ceva
~ simplitate in rezolvare: folosim \ la regresie
~ comparative cu toolbox-ul MATLAB: nlarx
~ cautam ordinul automat prin for-uri

MODEL OPTIM; na = nb = 6; m = 3;
MSE simulare validare = 0.658017;

MSE: Identificare - Predicție

	m=1	m=2	m=3
na=1	0.2369	0.1826	0.1793
na=2	0.2117	0.1606	0.1552
na=3	0.1907	0.1472	0.1398
na=4	0.1877	0.1410	0.1318
na=5	0.1875	0.1393	0.1291
na=6	0.1844	0.1338	0.1236
na=7	0.1839	0.1329	0.1216
na=8	0.1837	0.1324	0.1196

MSE: Identificare - Simulare

	m=1	m=2	m=3
na=1	1.5079	1.3548	1.4994
na=2	1.4259	0.9185	NaN
na=3	1.4519	0.7823	NaN
na=4	1.4377	1.2673	NaN
na=5	1.4436	NaN	1.0952
na=6	1.4371	NaN	0.6037
na=7	1.4301	NaN	0.6480
na=8	1.4224	NaN	0.6213

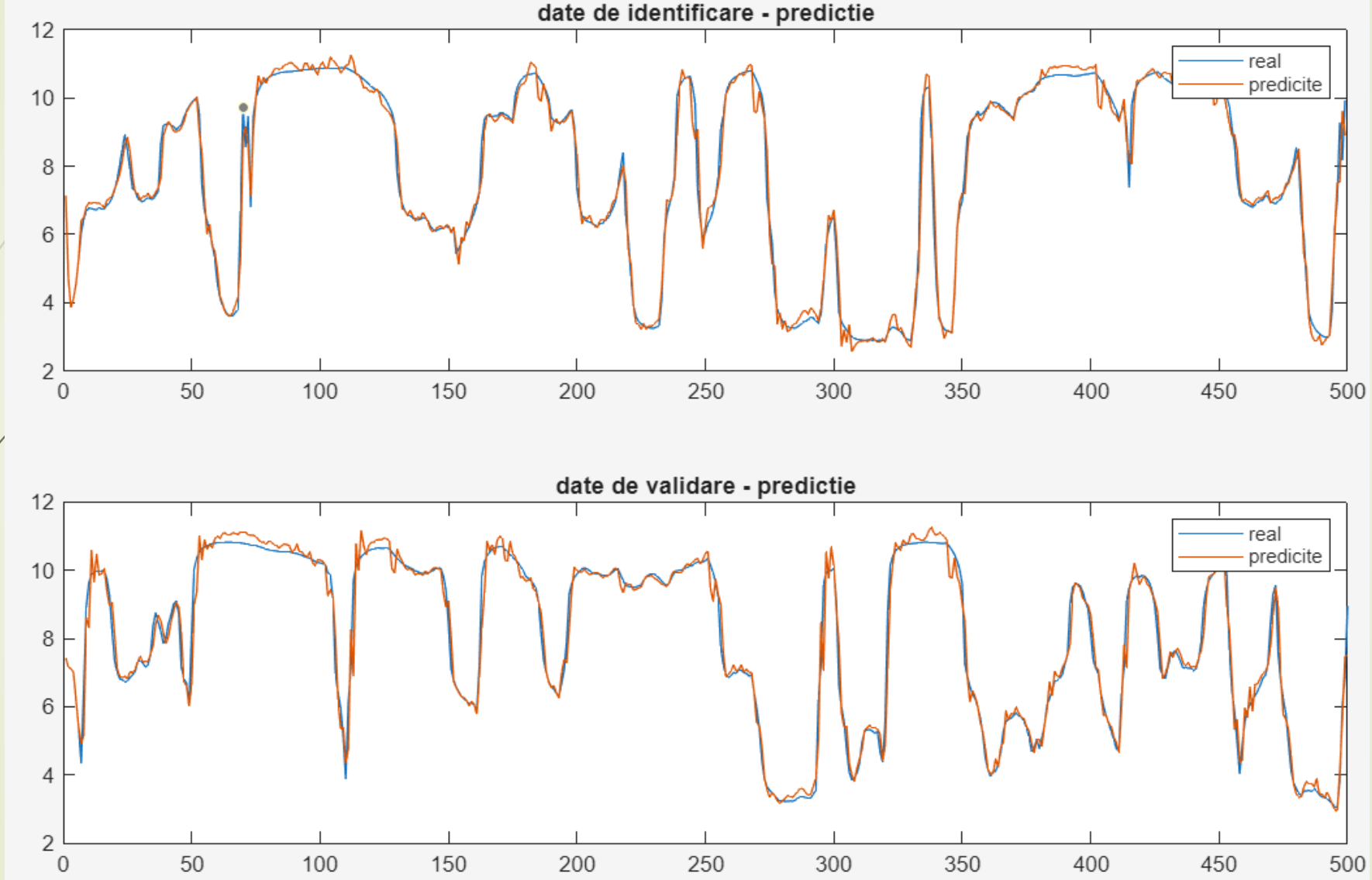
MSE: Validare - Predicție

	m=1	m=2	m=3
na=1	0.1770	0.1487	0.1423
na=2	0.1434	0.1426	0.1404
na=3	0.1386	0.1427	0.1452
na=4	0.1410	0.1547	0.1636
na=5	0.1404	0.1562	0.1689
na=6	0.1420	0.1613	0.1728
na=7	0.1411	0.1644	0.1849
na=8	0.1378	0.1590	0.1838

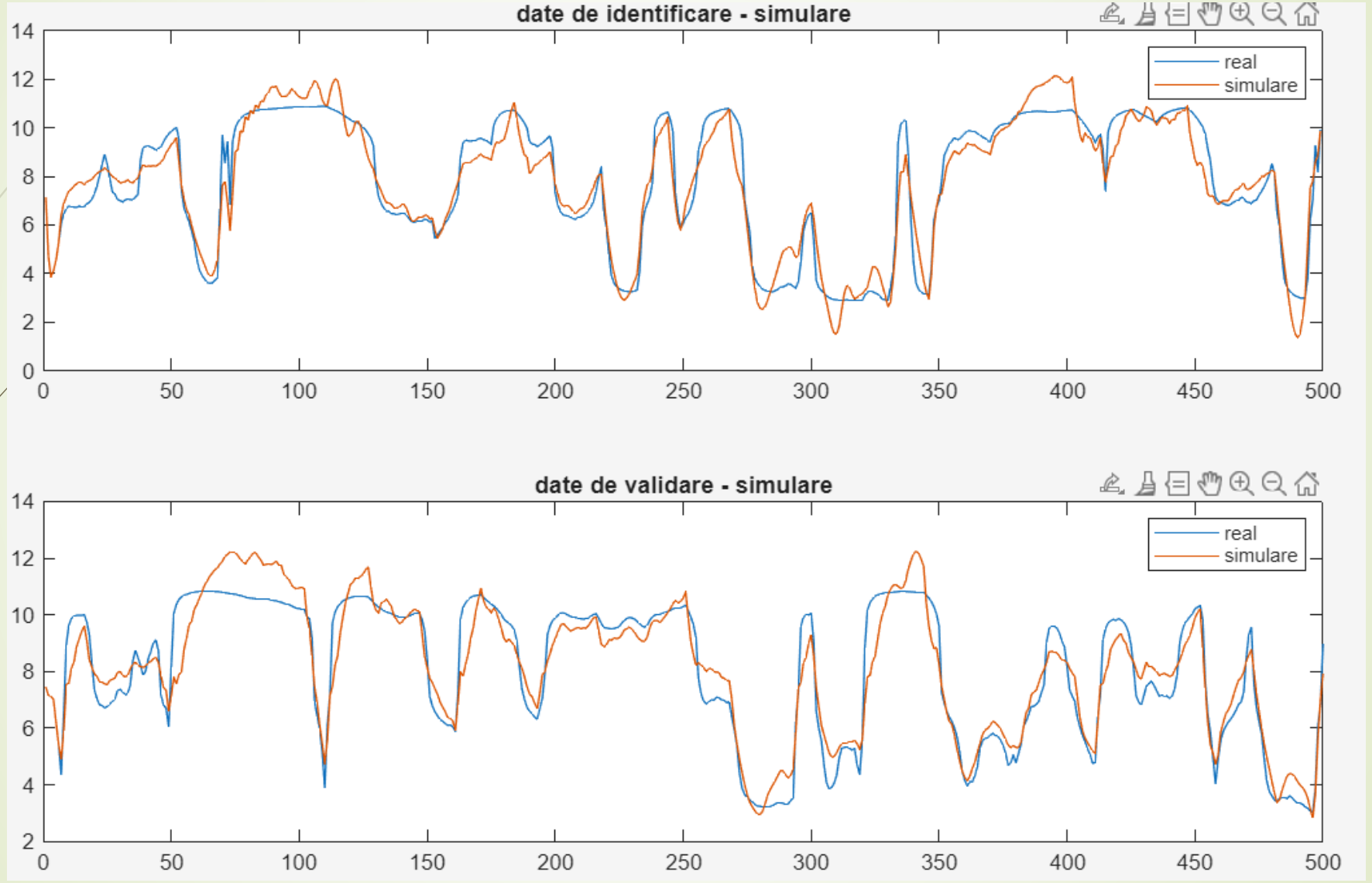
MSE: Validare - Simulare

	m=1	m=2	m=3
na=1	1.2360	NaN	2.4131
na=2	1.2298	1.0742	NaN
na=3	1.2641	0.7171	NaN
na=4	1.2706	0.7477	NaN
na=5	1.2834	NaN	NaN
na=6	1.2995	NaN	0.6580
na=7	1.3340	NaN	0.7339
na=8	1.2486	NaN	0.7036

Predictie


date de identificare - predictie
real
predicite
12
10
8
6
4
2
0
50
100
150
200
250
300
350
400
450
500
date de validare - predictie
real
predicite
12
10
8
6
4
2
0
50
100
150
200
250
300
350
400
450
500

Simulare


date de identificare - simulare
real
simulare
0
2
4
6
8
10
12
14
0
50
100
150
200
250
300
350
400
450
500
date de validare - simulare
real
simulare
2
4
6
8
10
12
14
0
50
100
150
200
250
300
350
400
450
500

Pentru modelul optim, erorile sunt:

na = nb = 6, m=3, nk=1

```
MODEL OPTIM SELECTAT
na = nb = 6
m  = 3
MSE predictie identificare = 0.123582
MSE predictie validare = 0.172790
MSE simulare identificare = 0.603721
MSE simulare validare = 0.658017
>>
```

Discutie a rezultatelor

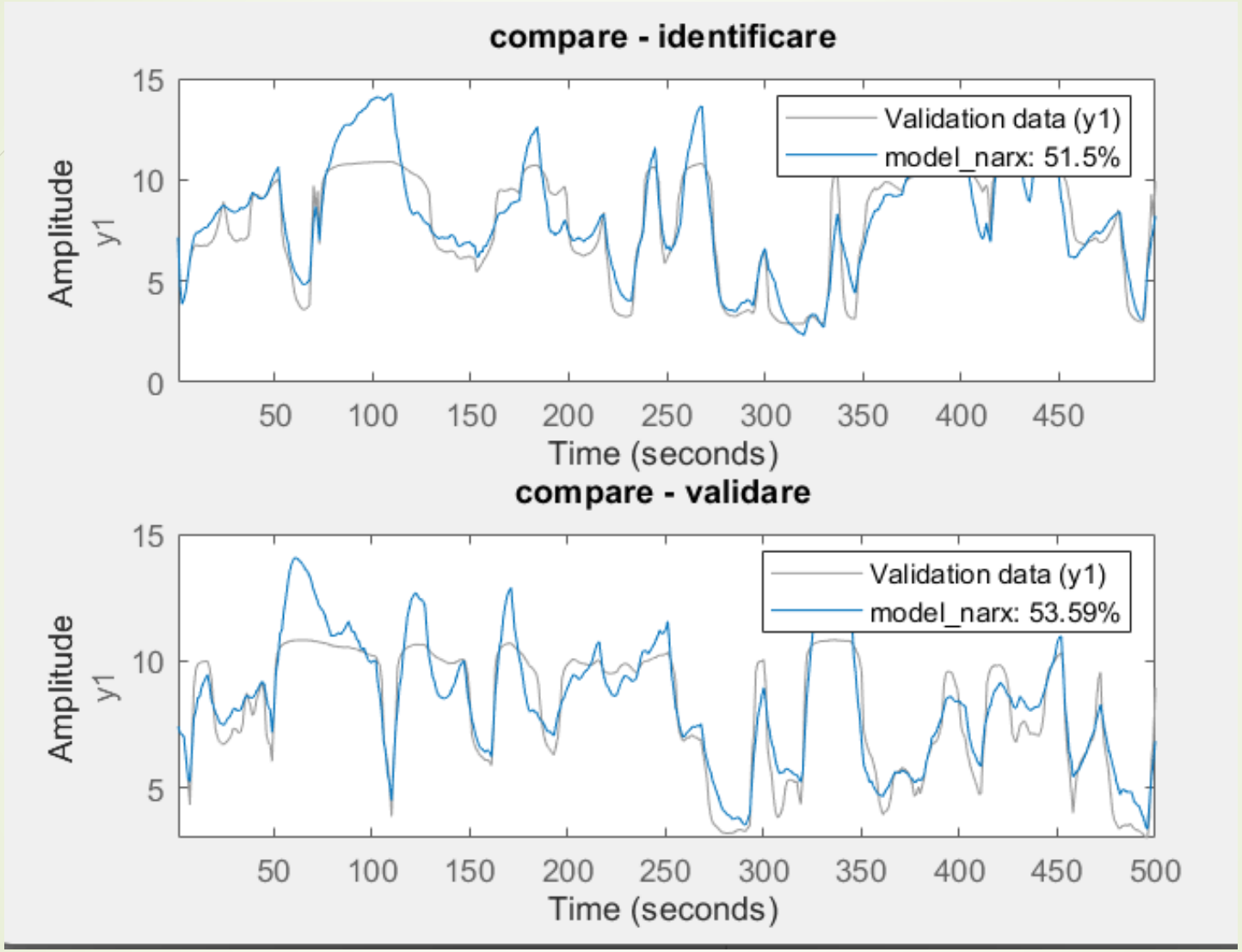
Calitatea modelului pe datele de identificare

Modelul NARX a fost identificat pentru diferite valori ale ordinelor na=nb și ale gradului de neliniaritate m, folosind drept criteriu de selecție **eroarea medie pătratică (MSE) la simulare pe setul de validare**, criteriu care evaluează cel mai corect capacitatea de generalizare a modelului.

Rezultatele arată că, pentru valori mici ale ordinului dinamic, eroarea de simulare este ridicată, ceea ce indică o descriere insuficientă a dinamicii sistemului. Pe măsură ce ordinul na $= \mathbf{nb}$ și gradul m cresc, calitatea simulării se îmbunătățește, iar MSE pe validare scade. Modelul optim obținut este caracterizat de:

$$n_a = n_b = 6, m = 3$$

pentru care se obține cea mai mică eroare de simulare pe setul de validare, fără apariția instabilităților. Compararea grafică (compare) arată că modelul urmărește corect dinamica ieșirii reale atât pe datele de identificare, cât și pe cele de validare.


compare - identificare
Amplitude
y1
15
10
5
0
50
100
150
200
250
300
350
400
450
Time (seconds)
Validation data (y1)
model_narx: 51.5%
compare - validare
Amplitude
y1
15
10
5
50
100
150
200
250
300
350
400
450
500
Time (seconds)
Validation data (y1)
model_narx: 53.59%

Cod sursa:

```
clc
clear all
close all
load("iddata-01.mat");

figure(1)
subplot(2, 1, 1), plot(id.u);
title('identificare - intrari');
grid on;
subplot(2, 1, 2), plot(id.y);
title('identificare - iesiri');
grid on;

figure(2)
subplot(2, 1, 1), plot(val.u);
title('validare - intrari');
grid on;
subplot(2, 1, 2), plot(val.y);
title('validare - iesiri');
grid on

na = 6;
nb = 6;
m = 3;
nk = 1;

%generare de regresori si aflare parametrii
phi_id = generareRegresori(id, na, nb, m);
Y = id.y(max(na, nb) + 1 : end);

theta = phi_id \ Y;

%predictie
y_pred_id = predictie(id.u, id.y, theta, na, nb, m);
y_pred_val = predictie(val.u, val.y, theta, na, nb, m);


figure(3)
subplot(2, 1, 1), plot(id.y); hold on;
plot(y_pred_id);
legend("real", "predicite");
title('date de identificare - predictie');


subplot(2, 1, 2), plot(val.y); hold on;
plot(y_pred_val);
legend("real", "predicite");
title('date de validare - predictie');


%simulare
y_sim_id = simulare(id.u, id.y, theta, na, nb, m);
y_sim_val = simulare(val.u, val.y, theta, na, nb, m);


figure(4)
subplot(2, 1, 1), plot(id.y); hold on;
plot(y_sim_id);
legend("real", "simulare");
title('date de identificare - simulare');


subplot(2, 1, 2), plot(val.y); hold on;
plot(y_sim_val);
legend("real", "simulare");
title('date de validare - simulare');
```

```
%compareare cu model narx
id_data = iddata(id.y, id.u);
val_data = iddata(val.y, val.u);


model_narx = nlarx(id_data, [na nb nk]);


figure(5)
subplot(2, 1, 1), compare(id_data, model_narx);
title('compare - identificare');
subplot(2, 1, 2), compare(val_data, model_narx);
title('compare - validare');


%testare


na_values = 1:8;
m_values = 1:3;
best_mse = inf;
best_theta = [];


MSE_pred_id = zeros(length(na_values), length(m_values));
MSE_sim_id = zeros(length(na_values), length(m_values));
MSE_pred_val = zeros(length(na_values), length(m_values));
MSE_sim_val = zeros(length(na_values), length(m_values));
```

```
for i_na = 1:length(na_values)
        na = na_values(i_na);
        nb = na; % pentru simplitate, conform cerintei
        for i_m = 1:length(m_values)
                m = m_values(i_m);

                phi_id = generareRegresori(id, na, nb, m);
                Y = id.y(max(na, nb)+1:end);
                theta = phi_id \ Y;

                y_pred_id = predictie(id.u, id.y, theta, na, nb, m);
                y_pred_val = predictie(val.u, val.y, theta, na, nb, m);

                y_sim_id = simulare(id.u, id.y, theta, na, nb, m);
                y_sim_val = simulare(val.u, val.y, theta, na, nb, m);

                % MSE
                MSE_pred_id(i_na,i_m) = mean((id.y - y_pred_id).^2);
                MSE_sim_id(i_na,i_m) = mean((id.y - y_sim_id).^2);
                MSE_pred_val(i_na,i_m) = mean((val.y - y_pred_val).^2);
                MSE_sim_val(i_na,i_m) = mean((val.y - y_sim_val).^2);

                % alegem simularea pe validare ca criteriu de selectie a
                        erorii
                % minime, intrucat este cea mai sensibila
                if MSE_sim_val(i_na, i_m) < best_mse
                        best_mse = MSE_sim_val(i_na, i_m);
                        best_na = na;
                        best_nb = nb;
                        best_m = m;
                        best_theta = theta;
                end
        end
end
```

```
titluri = {'Identificare - Predictie', 'Identificare - Simulare', ...
'Validare - Predictie', 'Validare - Simulare'};
matrici = {MSE_pred_id, MSE_sim_id, MSE_pred_val, MSE_sim_val};


for k = 1:4
        fprintf('\n==============================================\n');
        fprintf(' MSE: %s\n', titluri{k});
        fprintf('==============================================\n');
        % afisare cap de tabel (valorile lui m)
        fprintf('    ');
        for m = m_values
                fprintf(' m=%d ', m);
        end
        fprintf('\n');
        % afisare continut
        for i = 1:length(na_values)
                fprintf('na=%d  |', na_values(i));
                for j = 1:length(m_values)
                        %accesam matricea curenta matrici{k}
                        matrice_curenta = matrici{k};
                        fprintf('%9.4f ', matrice_curenta(i, j));
                end
                fprintf('\n');
        end
end
```

```
fprintf('\nMODEL OPTIM SELECTAT\n');
fprintf('na = nb = %d\n', best_na);
fprintf('m = %d\n', best_m);
fprintf('MSE predictie identificare = %.6f\n',
MSE_pred_id(best_na, best_m));
fprintf('MSE predictie validare = %.6f\n', MSE_pred_val(best_na,
best_m));
fprintf('MSE simulare identificare = %.6f\n', MSE_sim_id(best_na,
best_m));
fprintf('MSE simulare validare = %.6f\n', best_mse);

function y_pred = predictie(u, y, theta, na, nb, m)
        N = length(y);
        y_pred = y;

        for k = max(na,nb)+1:N
                d = [int(y,k,na); int(u,k,nb)];

                col = 1;
                val = 0;
                for i = 1:length(d)
                        for p = 0:m
                                for q = 0:(m-p)
                                val = val + theta(col) * d(i)^p * d(i)^q;
                                col = col + 1;
                                end
                        end
                end
                y_pred(k) = val;
        end
end
```

```
function y_sim = simulare(u, y, theta, na, nb, m)
        N = length(y);
        y_sim = y;

        for k = max(na,nb)+1:N
                d = [int(y_sim,k,na); int(u,k,nb)];

                col = 1;
                val = 0;
                for i = 1:length(d)
                  for p = 0:m
                    for q = 0:(m-p)
                      val = val + theta(col) * d(i)^p * d(i)^q;
                      col = col + 1;
                    end
                  end
                end
                y_sim(k) = val;
        end
end
```

```
function phi_data = generareRegresori(data, na, nb, m)
        u = data.u;
        y = data.y;
        N = length(y);

        k0 = max(na, nb) + 1;
        phi_data = zeros(N - k0 + 1, (m+1)*(m+2)/2 * (na+nb));

        row = 1;
        for k = k0:N
                y_int = int(y, k, na);
                u_int = int(u, k, nb);

                d = [y_int; u_int];
                col = 1;
                for i = 1:length(d)
                        for p = 0:m
                                for q = 0:(m-p)
                                        phi_data(row, col) = d(i)^p * d(i)^q;
                                        col = col + 1;
                                end
                        end
                end
                row = row + 1;
        end
end

function v = int(x, k, n)
        v = zeros(n,1);
        for i = 1:n
                if (k - i) > 0
                        v(i) = x(k - i);
                else
                        v(i) = 0;
                end
        end
end
```